Multiple Inheritance using interface :

In a class we have a constructor so, it is providing ambiguity issue but inside an interface we don't have constructor so multiple inheritance is possible using interface as shown in the program below.

The Implementer class constructor's super keyword will directly move to Object class constructor.

Example :


```
//Program
package com.ravi.ei;

interface Alpha
{
    void m1();
}
interface Beta
{
    void m1();
}

class Implementer implements Alpha, Beta
{
    @Override
    public void m1()
    {
      IO.println("Multiple inheritance is possible using interface");
    }
}

public class MIUsingInterface
{
    public static void main(String[] args)
    {
        Implementer i = new Implementer();
        i.m1();
    }
}
```

Extending an interface :

One interface can **extend another interface** but it **can't** implement an interface as shown in the program.

```
package com.ravi.interface_ex;

interface A
{
    void m1();
}
interface B extends A
{
    void m2();
}

class Implementer implements B
{
    @Override
    public void m1()
    {
      IO.println("A1 method is overridden");
    }

    @Override
    public void m2()
    {
      IO.println("A2 method is overridden");
    }
}
public class ExtendingInterface
{
    public static void main(String[] args)
    {
        Implementer i = new Implementer();
        i.m1();
        i.m2();

    }

}
```

Java 8 features [18th March 2014]

Limitation of Abstract method :

OR

Maintenance problem with interface in an Industry upto JDK 1.7

The major maintenance problem with interface was, if we add any new abstract method at the later stage of development inside an existing interface then all the implementer classes have to override that abstract method otherwise the implementer class will become as an abstract class so it is one kind of boundation.

We need to provide implementation for all the abstract methods available inside an interface whether it is required or not?

To avoid this maintenance problem java software people introduced default method inside an interface.

What is a default method [Backward Compatibility] :

It is possible to write default method (default keyword + method body) inside an interface from JDK 1.8V to provide BACKWARD COMPATIBILITY. [Without modifying the implementer classes (No compilation error)]

A default method we can declare only inside an interface but not in a class.

A default method must be declared with default keyword and contains method body.

By default access modifier of a default method is public.

A default method cannot be marked as abstract, final or static.

A default method may be overridden by an implementer class that implements the interface.[Without any boundation]

It is mainly used to provide "DEFAULT IMPLEMENTATION"

```
//Program.
package com.ravi.java_8_features;

public interface Vehicle
{
    void run();
    void horn();

    //Java 8 default method  //Backward Compatibility
    default void digitalMeter()
    {
        IO.println("Default implementation of Digital Meter");
    }
}

package com.ravi.java_8_features;
public class Car implements Vehicle
{
    @Override
    public void run()
    {
        IO.println("Car is running");
    }

    @Override
    public void horn()
    {
        IO.println("Car has horn");
    }

    @Override
    public void digitalMeter()
    {
        IO.println("Car has Digital Meter facility");
    }
}

package com.ravi.java_8_features;

public class Bike implements Vehicle
{
    @Override
    public void run()
    {
        IO.println("Bike is running");
    }

    @Override
    public void horn()
    {
        IO.println("Bike has horn");
    }
}

package com.ravi.java_8_features;

//All
public class DefaultMethod
{
    public static void main(String[] args)
    {
        Vehicle vehicle = null;
        vehicle = new Car();vehicle.run();vehicle.horn();vehicle.digitalMeter();
        vehicle = new Bike();vehicle.run();vehicle.horn();
    }

}
```

What was the solution given by sun microsystem to add a new abstract method inside an existing interface :

```
interface Vehicle
{
  void run();
}

class Car implements Vehicle
{
  @Override
  public void run()
  {
  }
}
```

```
interface Battery extends Vehicle
{
  void battery();
}
class BatteryCar extends Car implements Battery
{
  @Override
  public void battery()
  {
  }
}
```

Priority in between deafult and concrete method :

While working with class and interface, default method is having low priority than concrete method, In the same way class is more powerful than interface.

interface A

{

}

class B

{

}

class C extends B implements A {} //Valid

class C implements A extends B {} //Invalid

```
package com.ravi.java_8_features;

interface Alpha
{
    default void m1()
    {
      IO.println("Default Method of Alpha interface");
    }
}
class Beta
{
    public void m1()
    {
      IO.println("Concrete Method of Beta class");
    }
}
class Implementer extends Beta implements Alpha //it extends then implements
{

}
public class PriorityDemo
{
    public static void main(String[] args)
    {
        Implementer i = new Implementer();
        i.m1();
    }
}
```